exists – `$counter` only initializes the first time the function is called – so we don't need the `isset()` test. Secondly, `$counter` is still only available within the local scope of `getCount()`. This is why we are warned when we try to access it from global scope.

By using `static` in this way we have avoided a global `$count` variable. Imagine that we needed several different counting functions like this in our program, we'd soon end up with lots of ugly variable names like `$count_1`, `$otherCount`, `$countThis`, `$countThat`, and so on to avoid functions stepping on each other's data.

Scope Persists Across Files

Scope boundaries only exist when control passes into or out of a function. When code is spread across multiple files, scope is inherited from the calling code when a file is incorporated into the program via `include`, `require`, etc. Dividing a program into multiple source code files does not generate scope:

```
// include_scope0.php
$song = 'Run to the Hills';

include __DIR__ . "/include_scope1.php";

// include_scope1.php
print $song . PHP_EOL;

Output (include_scope0.php):
Run to the Hills
```

We have two source code files, `include_scope0.php` and `include_scope1.php`. We execute `include_scope0.php`, which defines a global variable `$song` and then tells PHP to read in the second file via an `include` statement. The second file inherits whatever scope is active when PHP reaches `include __DIR__. '/include_scope1.php'` – in this case, global scope. This means that `$song` is available during execution of `include_scope1.php` and the `print` statement succeeds.

What if we include a file as part of a function definition? Well, instead of inheriting global scope, the included file inherits the scope of the function body at the point the PHP engine hits the `include` statement:

```
// include_scope2.php
$song = 'Hallowed Be Thy Name';
```